MoveInSync Hierarchical Configuration Platform

Case Study 2: System Architecture, Concurrency Design & Production Implementation

Author: Saksham Gupta (Reg No: 230911186) • B.Tech IT • saksham77779@gmail.com

Code Repository: GitHub • System Walkthrough: Demo Video

1. Operational Problem Statement & Context

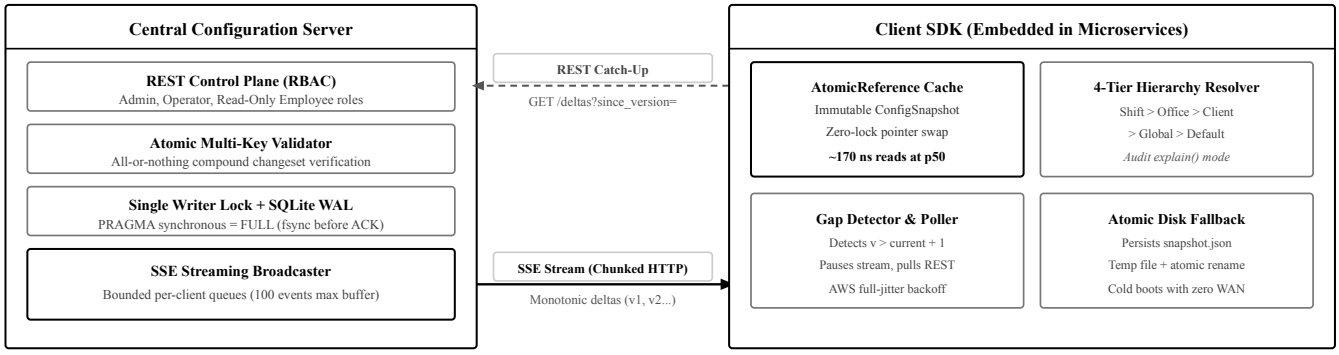
MoveInSync manages enterprise employee commute transport across 100+ corporate clients (e.g., Google, Microsoft, and Amazon campuses), coordinating tens of thousands of shared cab trips daily. The backend infrastructure comprises 40+ microservices spanning trip booking, route optimization, vehicle dispatch, billing, and female employee night-safety compliance. Every dispatch decision evaluates multi-level policies—including `booking_cutoff_minutes`, `max_route_deviation`, `night_escort_required`, and allowed vehicle tiers.

Executing synchronous remote HTTP calls to a centralized database during high-frequency dispatch evaluations introduces intolerable RPC latency and creates a catastrophic single point of failure. The platform mandates two opposing engineering guarantees: **sub-microsecond local reads** (measured ~170 ns at p50) directly from process memory with zero thread lock contention, and **sub-second fleet propagation** (< 1,000 ms SLA; measured ~320 ms) when administrators modify policies, perform emergency overrides, or execute staged regional canary rollouts.

2. System Architecture & Component Topology

The system strictly decouples the high-throughput read plane from the administrative write plane. Writes execute through a centralized control plane with full ACID durability, while reads execute lock-free within client microservices:

- **REST Control Plane & Compound Validator:** Spring Boot 3 control service providing versioned administrative endpoints with RBAC (Admin, Operator, Read-Only). Validates multi-key changesets atomically before committing, ensuring partial invalid payloads (e.g. negative cutoffs) are rejected with HTTP 400 Bad Request.
- **Durable Storage Engine (SQLite WAL):** Single-writer architecture guarded by a reentrant mutex and backed by SQLite configured with Write-Ahead Logging (WAL) and `PRAGMA synchronous = FULL`. Changesets increment a monotonic global version and fsync to disk before returning an HTTP 201 ACK, surviving process crashes without data corruption.
- **Streaming SSE Broadcaster:** Maintains persistent HTTP/1.1 chunked Server-Sent Events streams to active client nodes. Implements backpressure via bounded per-client queues (100 events max) to protect heap stability against slow consumer nodes.
- **Embedded Client SDK:** Java library embedded directly into commute microservices. Caches the full configuration graph in an immutable `ConfigSnapshot` wrapped in an `AtomicReference`, tracks sequence continuity, and persists atomic disk snapshots (`snapshot.json`) to guarantee instant cold boot during WAN partitions.



3. Data Flow & Separation of Planes

Write Pipeline: Administrative requests submit changesets via `POST /api/v1/config/changeset`. The control plane validates business constraints, acquires the reentrant write lock, increments the monotonic version, appends to the SQLite Write-Ahead Log, and invokes `PRAGMA synchronous = FULL`. Only after the WAL entry is fsynced to disk does the server return HTTP 201 Created and emit a versioned delta to the SSE event loop.

Read & Ingestion Pipeline: Subscriber microservices receive the delta event over the persistent chunked SSE stream. The SDK verifies monotonic continuity, clones the existing configuration graph, applies the overrides, and points the `AtomicReference` to the new immutable snapshot. Client worker threads resolve keys against this local snapshot in memory with zero network latency, zero lock synchronization, and zero contention.

4. Concurrency Model & Lock-Free Read Path

Commute microservices frequently evaluate compound keys during dispatch decisions. For instance, before confirming a cab booking, the routing engine simultaneously queries `booking_cutoff_minutes` and `cancellation_window`. In a naive implementation using a mutable synchronized map, an update arriving mid-evaluation risks exposing a mixed-version state where one key reflects version 42 and the other reflects version 43. Introducing reader-writer locks resolves consistency but degrades throughput via lock acquisition overhead and contention under heavy traffic.

To solve this, the SDK maintains state inside an immutable `ConfigSnapshot` wrapped in a Java `AtomicReference`. Worker threads capture the snapshot pointer once per evaluation:

```
ConfigSnapshot snapshot = client.getSnapshot(); int cutoff = client.get(snapshot,
"booking_cutoff_minutes", Integer.class); int window = client.get(snapshot, "cancellation_window",
Integer.class);
```

All subsequent key lookups execute strictly against that immutable snapshot instance. Reads bypass synchronization barriers, taking ~170 ns at p50 (~5.4 million ops/sec per thread). When incoming SSE deltas arrive, the background subscriber thread builds a cloned, updated snapshot and swaps the pointer via a single atomic `set()` call (< 10 ns). In stress testing with 50 concurrent reader threads performing 2,100,000 evaluations during continuous writes, zero torn reads or thread deadlocks occurred.

5. Hierarchical Precedence & Explain Audit Trail

Configurations resolve through a deterministic 4-tier hierarchy: **Shift > Office > Client > Global > Default**. The resolver evaluates scoped paths in `O(1)` time against pre-indexed snapshot scopes. For compliance and operations auditing, the SDK exposes an `explain(key, context)` interface that returns the full resolution lineage—showing exactly which scope provided the winning value, which higher-level values were overridden, and the authoritative timestamp.

For example, querying `explain("booking_cutoff_minutes", ctx)` returns a structured trace: `{"resolved": 45, "scope": "OFFICE:BLR_CAMPUS", "overridden": [{"scope": "CLIENT:MSFT", "val": 30}, {"scope": "GLOBAL", "val": 60}], "version": 44}`. This audit trail is critical for transport operations teams investigating dispatch anomalies or employee escalation tickets.

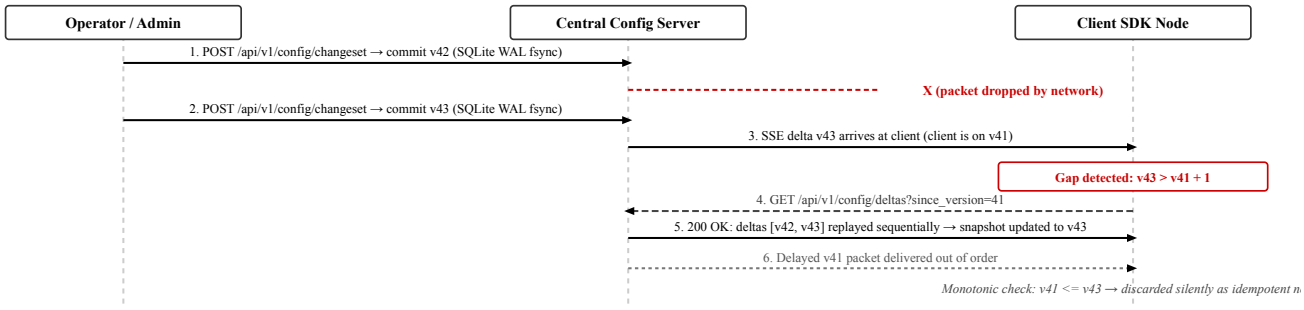
6. Staged Canary Rollouts & False-Gap Mitigation

In production, MoveInSync operators roll out operational policy changes incrementally—for example, piloting a new 45-minute booking cutoff in the Bangalore campus before enabling it in Hyderabad. In an early implementation, assigning version 44 to the Bangalore-only update caused Hyderabad SDK nodes listening to the global SSE stream to observe an apparent sequence gap from version 43 to 45. The gap detector reacted by triggering an immediate wave of HTTP REST queries (`GET /deltas?since_version=43`) across hundreds of nodes, generating a self-inflicted thundering herd.

This was resolved by implementing **empty delta propagation**. When a staged change commits at version 44, the server emits the targeted changeset to Bangalore subscribers, while emitting an empty delta payload (`empty=true`) to non-targeted subscribers. Hyderabad nodes ingest the empty delta and advance their version watermark to 44 without altering cached values. This preserves strict fleet-wide version alignment and eliminates false gap storms while keeping rollout isolation fully intact.

7. End-to-End Propagation & Resynchronization Flow

The lifecycle below illustrates how administrative changes commit durably, stream to client nodes, and recover deterministically from transient network packet loss:



Reconnection Resilience & Jitter Protocol: When network drops occur, the SDK applies truncated exponential backoff with full decorrelated jitter: `t_sleep = min(2000 ms, random(50 ms, t_prev × 3))`. This prevents synchronized reconnect storms across the 200 microservice nodes from overwhelming the central server upon recovery. Missing deltas are pulled via REST catch-up and applied in strict sequence order before live SSE streaming resumes.

8. Fault Tolerance & Failure Recovery

Table 1: Fault Scenarios, Observable Symptoms, and Recovery Mechanisms

Failure Scenario	Observable Service Symptoms	Architectural Recovery Mechanism
Server Crash / Hard kill -9	In-flight writes receive HTTP 500; reads continue from in-memory cache without disruption.	SQLite WAL ensures crash durability (PRAGMA synchronous=FULL); recovers on reboot with zero corruption.
Network Packet Loss & Version Gap	Client on v41 receives v43 over SSE; gap detector flags missing intermediate delta.	SDK buffers v43, queries `GET /deltas?since_version=41`, replays missing deltas, and updates snapshot.
Stale Out-of-Order Delta	Delayed network routing delivers v41 after client is already active on v43.	Monotonic check (`version <= currentVersion`) silently discards the packet as an idempotent no-op.
Invalid Compound Changeset	Operator submits 5 keys, but 1 key violates validation (e.g., negative cutoff duration).	Atomic multi-key validator aborts transaction with 400 Bad Request; zero keys mutate in database.
Slow SSE Subscriber Node	Consumer node reads events slower than server broadcast production rate.	Bounded queue (100 events) overflows; server evicts client to protect heap; client reconnects via REST.
Cold Boot in Isolated Partition	Microservice container starts during WAN partition without central server access.	SDK hydrates immediately from local `snapshot.json` written atomically via temp-file swap.

9. Performance Benchmarks & Empirical Latency Profile

Benchmarks executed on OpenJDK 17 LTS (Apple M1, 8 cores, 8 GB RAM) following 50,000 warm-up iterations to ensure HotSpot C2 JIT compilation.

Table 2: Empirical Latency and Throughput Benchmarks

Operation	p50	p90	p99	Max	Throughput / Scope
In-Memory 4-Tier Read	170 ns	208 ns	250 ns	412 ns	5.4M ops/sec / thread
Fleet Propagation (200 Nodes)	278 ms	320 ms	320 ms	345 ms	200 clients in 1 JVM
Single-Writer Commit (WAL fsync)	0.38 ms	0.52 ms	0.85 ms	1.20 ms	~2,500 writes/sec max

Read latency is sub-microsecond because it avoids lock synchronization and network I/O. Propagation latency is bounded by SQLite fsync and thread scheduling across 200 concurrent client threads within the test JVM.

10. Architectural Trade-Offs & Design Boundaries

- **SQLite WAL vs Distributed Raft:** For 99.99% local reads and low write volume (< 10/min), embedded SQLite WAL with synchronous fsync avoids split-brain and consensus churn (~2,500 writes/sec capacity). Multi-region scaling utilizes PostgreSQL read replicas.
- **Monotonic Versioning vs CRDTs:** Corporate commute policies follow top-down governance (Shift > Office > Client > Global > Default). Monotonic versioning provides deterministic ordering without CRDT metadata bloat or tombstone overhead.
- **SSE Push vs Periodic Polling:** Polling across 40 services generating 2,400 queries/min wastes compute and averages 2.5s update lag. SSE push eliminates idle database queries and guarantees sub-second updates (~320 ms).
- **SSE vs WebSockets:** Configuration delivery is strictly unidirectional (server to client). SSE runs over HTTP/1.1 chunking, easily traversing enterprise API gateways, proxies, and firewalls without custom framing.

11. Verification Suite & Test Matrix

The project includes **26 automated tests** across three Maven modules, running in 34 seconds via `./mvnw test` with 100% pass rate:

- **config-core (11 tests):** 4-tier hierarchy precedence, audit trail `explain()` metadata, negative duration rejection (-30 min cutoff), and time window overlap constraints.
- **config-client-sdk (2 tests):** Zero-lock snapshot reads, canary empty delta ingestion, and offline disk snapshot hydration.
- **config-server (13 tests):** Compound all-or-nothing validation, 50-thread concurrent atomic reads, canary rollouts, forward rollbacks, multi-tenant REST/SSE isolation, 401 unauthorized rejection, SSE gap catch-up, stale packet drop, `kill -9` crash durability, and 200-client reconnect benchmark.